PLATOLIVES

Version 5.0.0 User and Technical Manual

Native multi-platform PLATO terminal

Connection and keyboard profiles | State-aware scripting | Physical display engines

Fabio Montarsolo • October 2026

Contents

Contents 2

About This Manual 6

Acknowledgements . . . 6

Contents 7

1. Why PlatoLives Exists 8

1.1 Preservation through observable behavior . . . 8

1.2 What makes PlatoLives distinctive . . . 8

2. Design Principles and Architecture 9

2.1 Logical and optical layers . . . 9

2.2 Platform model . . . 9

3. Getting Started and Connection Profiles 10

3.1 First connection . . . 10

3.2 Connection profiles . . . 10

3.3 Connection lifecycle and recovery . . . 10

3.4 Multiple sessions . . . 10

4. Keyboard Profiles and Input 11

4.1 A keyboard that belongs to the session . . . 11

4.2 Standard profile at a glance . . . 11

4.4 Touch and clipboard . . . 11

5. Scripting and Session Automation 12

5.1 Automation that reads the terminal . . . 12

5.2 Reading, deciding and acting . . . 12

5.3 Connection-aware recovery . . . 12

5.4 Language essentials . . . 12

5.6 Example: adaptive login and recovery . . . 13

5.7 Script tools . . . 13

6. Rendering Engines and Display Controls 14

6.1 Choosing a display engine . . . 14

6.2 Display controls . . . 14

6.3 Performance HUD . . . 14

7. MicroTutor, Audio and Platform Support 15

7.1 Offline MicroTutor . . . 15

7.2 Audio . . . 15

7.3 TrueType rendering . . . 15
7.4 Supported builds . . . 15

8. Menus and Everyday Reference **16**
C.1 Connection menu . . . 16
C.2 Connection Profiles Manager . . . 16
C.3 Keyboard Profiles Manager . . . 16
C.4 Keyboard Reference . . . 17
C.5 Script Manager and scripting reference . . . 17
C.6 Live Text Buffer and copy tools . . . 17
C.7 Diagnostic tools . . . 18
C.8 Performance and Vector HUD . . . 18
C.9 View, Micro Tutor and platform-specific tools . . . 18

9. Diagnostics, Performance and Troubleshooting **19**
9.1 Diagnostic logging . . . 19
9.2 Performance strategy . . . 19
9.3 Troubleshooting . . . 19

10. Technical Architecture and Build Notes **20**
10.1 Shared core . . . 20
10.2 Transport . . . 20
10.3 Versioning and builds . . . 20
10.4 Configuration storage . . . 20

11. License, Credits and References **21**
Version 5.0.0 release summary . . . 21

Appendix A. Real Plasma Rendering: Technical Design **22**
A.1 Historical and logical model . . . 22
A.2 Four-by-four cell footprint . . . 22
A.3 Neon persistence . . . 22
A.4 Dual separable blur . . . 22
A.5 Density fusion and orange response . . . 22
A.6 Distortion and touch . . . 22

Appendix B. Real Color CRT and Optical Geometry: Mathematical Design **23**
B.1 Three-tap Gaussian electron beam . . . 23
B.2 Pure chromatic shadow mask . . . 23
B.3 Soft analog scanlines . . . 23
B.4 Calibrated forward distortion . . . 23
B.5 Inverse touch mapping . . . 23
B.6 Continuous P22 color-phosphor decay . . . 23

B.7 Additive halation and profile composition 23

Appendix C. The Tektronix 4014 Optical Engine & 24

25

C.1. Operating Physics of the Direct-View Bistable Storage Tube (DVBST) 25

C.1.1. The Dual-Gun Electron Optics 25

C.1.2. Secondary Emission and Dielectric Bistability 25

C.1.3. Deflection Settling Times and Vertex Dwell 26

C.1.4. The Page Erase Flash (Bulk Dielectric Discharge) 26

C.2. Technical Challenges in Achieving Believable Simulation 26

C.2.1. Vector Continuity vs. Discrete Pixel Matrices 27

C.2.2. Transient Optical Dynamics (The Phosphor Bloom Paradox) 27

C.2.3. The Baud-Rate Serial Bottleneck 27

C.3. PLATO vs. Tektronix: Architectural and Semantic Divergence 27

C.3.1. The PLATO Plasma Cell Mechanics 27

C.3.2. The DVBST Erase Impossibility 28

C.3.3. The "Display List" Fallacy 28

C.4. The PlatoLives Optical Engine Architecture 28

C.4.1. Dual-Grid Spatial Decoupling (tek_stored & tek_flare) 29

C.4.2. Continuous Sub-Pixel Splatting & Settling Emulation 29

Dwell Spot Injection 29

C.4.3. The Erase Resolution: Trench Carving vs. Canonical Resync 29

1. Geometric Vector Erase (Destructive Trench Carving) 29

2. Text and Block Resync (plato_tek_resync_box_from_fb) 29

C.4.4. Hardware Serial Pacer (Token-Bucket Flow Control) 30

C.4.5. On-the-Fly Non-Linear Chromatic Synthesis 30

C.5. Architectural Conclusions & Verification 31

Appendix D. Historical and Technical References 32

NATIVE MULTI-PLATFORM PLATO TERMINAL | CONNECTION AND KEYBOARD PROFILES | STATE-AWARE SCRIPTING | PHYSICAL DISPLAY ENGINES 512 x 512 logical display | 2048 x 2048 optical rendering | macOS, Windows and Linux

About This Manual

This manual describes PlatoLives 5.0.0 as both a user application and a preservation-oriented terminal implementation. The user chapters explain the ideas and workflows that distinguish PlatoLives: reusable connection profiles, configurable keyboard profiles, state-aware scripting, offline MicroTutor operation, and a family of display engines that reproduce the visual behavior of historical terminals without changing the PLATO logical contract.

Implementation details are intentionally concentrated in the technical chapters. Everyday sections explain what a feature does, when to use it, and how it interacts with saved profiles. The third rendering-engine technical section, , is reproduced verbatim from its supplied source.

Acknowledgements

PlatoLives gratefully acknowledges Tom Hunter, Paul Koning, Joe Stanton, Bill Galcher, Steve Zoppi, Dale Sinder, the Cyber1 community, lesson authors, archivists, testers, and everyone who preserved practical knowledge of PLATO protocols and terminal behavior. Pterm is acknowledged as an important source of historical and behavioral knowledge, including PTAT keyboard translation, classic typography, protocol decoding, and live-host edge cases.

The local MicroTutor environment incorporates the open-source z80emu processor core by Lin Ke-Fong. TrueType rasterization uses stb_truetype by Sean T. Barrett. PlatoLives thanks both authors for making these components available.

Contents

User guide

Why PlatoLives Exists
Design Principles and Architecture
Getting Started and Connection Profiles
Keyboard Profiles and Input
Scripting and Session Automation
Rendering Engines and Display Controls
MicroTutor, Audio and Platform Support
Menus and Everyday Reference
Diagnostics, Performance and Troubleshooting
Technical Architecture and Build Notes
License and Credits

Technical appendices

Appendix A. Real Plasma Rendering: Technical Design Appendix B. Real Color CRT and Optical Geometry: Mathematical Design Appendix C. The Tektronix 4014 Optical Engine & Vector Architecture Appendix D. Historical and Technical References

1. Why PlatoLives Exists

1.1 Preservation through observable behavior

PlatoLives is a native, multi-platform terminal for PLATO IV, Cyber1 and compatible services. Its purpose is not merely to decode a historical byte stream. It preserves the observable behavior of several terminal families while providing a practical modern environment for connection management, keyboard adaptation, automation and local learning software.

The protocol, character sets, graphics operations, touch semantics and 512 x 512 coordinate system remain the historical contract. Networking, memory organization, graphics APIs, configuration storage and user-interface management are modern implementation choices.

Design position: historical fidelity belongs in observable terminal behavior; modern engineering belongs in the implementation.

1.2 What makes PlatoLives distinctive

Connection profiles combine host, port, display engine and session preferences in reusable named configurations.
Keyboard profiles adapt PLATO's specialized keyboard to modern keyboards and different styles of use without changing the host protocol.
State-aware scripting reacts to text found on the terminal screen and can recover, reconnect or switch connection targets.
Multiple rendering engines separate the authoritative logical terminal from optical presentation: crisp diagnostic views, gas-plasma, color CRT and Tektronix DVBST.
Local MicroTutor support runs compatible .mte disk images offline through an integrated Z80 environment.
Native front ends present the same shared terminal core through macOS, Windows and Linux technologies.

2. Design Principles and Architecture

Principle	Meaning in PlatoLives 5.0.0
Fidelity	Preserve host-visible protocol, graphics, character, keyboard and touch behavior.
Separation	Keep the 512 x 512 logical terminal authoritative and treat optical rendering as a presentation stage.
Profiles	Save connection, keyboard and display choices as named, reusable working contexts.
Automation	Let scripts inspect real screen state rather than blindly replaying timed keystrokes.
Portability	Share the pure C11 terminal core and isolate platform-specific presentation and dialogs.
Resilience	Keep connection attempts asynchronous, cancellable and recoverable.
Restraint	Reproduce optical characteristics without exaggerating curvature, bloom or persistence.
Verification	Provide crisp modes, logging, screenshots, text extraction and telemetry for diagnosis.

2.1 Logical and optical layers

The logical display has 512 x 512 PLATO coordinates, a 64 x 32 text matrix, canonical M0/M1 fonts and programmable M2/M3 fonts. Monochrome occupancy and color state are maintained independently from the optical engines. Consequently, selecting Plasma, CRT or Tektronix changes presentation, not the host-visible coordinate system.

2.2 Platform model

PlatoLives uses a shared C11 core with native presentation layers: Cocoa/Core Animation and a Metal-aligned BGRA surface on macOS, Win32 and Direct3D 11 on Windows, and SDL2/OpenGL 3.3 on Linux desktop. A Linux console build provides a headless ANSI TrueColor environment. Version 5.0.0 also supports native Windows ARM64 builds.

3. Getting Started and Connection Profiles

3.1 First connection

The supplied default profile connects to Cyber1 at cyberserv.org on TCP port 8005. Start PlatoLives, choose the profile, and allow the terminal identification exchange to complete. Return/Enter sends PLATO NEXT by default. The public demonstration sequence is typically:

Press Return at the opening prompt.
Enter demo1, then press Return.
Enter demo, then send SHIFT-STOP.
At the password prompt, press Return for the public account.

Host prompts may change. A saved or built-in script is preferable for repeated sessions because it can check the actual text displayed before acting.

3.2 Connection profiles

The Connection Profiles Manager is the place where a recurring terminal session is given a stable identity. Instead of choosing a host and then rebuilding the same visual environment at every launch, the user names the session and saves its network destination together with the presentation choices that belong to it. A Cyber1 profile may open in Real Plasma with restrained persistence; a development host may use Crisp Monochrome for exact diagnostics; a demonstration profile may select Real Color CRT and full screen. The profile therefore describes the working context, not merely the socket address.

The manager is organized around a list of profiles and a single detail pane. Selecting a profile displays its host, port, preferred rendering engine, optical settings, full-screen choice and optional startup script. Adding or duplicating a profile creates an independent record, so experimental changes do not disturb a known-good connection. Deleting a profile removes only that saved definition; it does not alter scripts or keyboard profiles. One profile may be marked as the startup default.

Only controls that make sense for the chosen display engine are shown. Plasma persistence is not presented while editing a crisp profile, CRT beam controls belong only to Real Color CRT, and Tektronix serial speed belongs only to the storage-tube engine. This conditional design prevents contradictory settings while preserving each engine's calibration when the user moves between modes.

Saving records the changes. Connecting applies the selected record to the current terminal window: the previous transport is closed cleanly, protocol and optical state are reset, and the new session begins with the profile's own presentation and startup automation. Existing windows remain independent, which makes it practical to keep multiple hosts or renderers open side by side.

3.3 Connection lifecycle and recovery

Connection attempts are asynchronous. The terminal remains responsive while DNS resolution and the TCP handshake are in progress, and Escape cancels an unwanted attempt. A failure is reported in a native dialog with a concise reason and the choice to retry or close. Reconnect repeats the most recent endpoint; Switch Profile deliberately changes the complete working context. Scripts use the same connection model through connect, connect default, connect last, reconnect, and disconnect, so interactive and automated sessions recover in the same way.

3.4 Multiple sessions

Each terminal window owns an independent transport, terminal state, logical display, optical buffers and timing state. This permits simultaneous sessions using different hosts, profiles, display engines or scripts.

4. Keyboard Profiles and Input

4.1 A keyboard that belongs to the session

PLATO's keyboard is part of the terminal, not an incidental collection of PC shortcuts. Keys such as NEXT, BACK, STOP, LAB, DATA, SUPER, SUB and ACCESS carry meanings that ordinary desktop keyboards do not express directly. PlatoLives therefore places translation in named keyboard profiles. The host still receives canonical PLATO key codes; the profile determines how the local keyboard produces them.

The Keyboard Profiles Manager follows a learn-and-review workflow. The user starts from a supplied profile or duplicates one, selects a PLATO action, and presses the physical key or combination to bind. The complete mapping remains visible as a coherent layout rather than a sequence of unrelated shortcut fields. A binding may emit a named PLATO key, a raw byte or a short PLATO sequence. Profiles are stored as portable JSON records and can be changed without reconnecting.

The supplied Standard PLATO profile is intended for lessons and general terminal work. WASD Gaming provides familiar movement, WASX / Avatar follows the classic forward/left/right/back arrangement, WADX forms the directional diamond used by several PLATO/TUTOR programs, and Numpad Arrows supports eight-way movement. These are starting points, not mandatory conventions. A personal profile can preserve the controls of a particular game or accommodate a compact keyboard while leaving all other sessions untouched.

A short profile-cycle shortcut, Cmd+Shift+G on macOS or Ctrl+Shift+G on Windows and Linux, is useful when moving between ordinary terminal work and a game. The Keyboard Reference window documents the fixed default shortcuts; custom bindings are inspected in the Keyboard Profiles Manager. The traditional default mappings are summarized below for orientation, not as a substitute for the profile manager.

4.2 Standard profile at a glance

PLATO action	Usual physical key	Shifted or alternate form
NEXT / SHIFT-NEXT	Return	Shift+Return or double Return
BACK / SHIFT-BACK	Escape or F8	Shift+Escape, Shift+F8 or double F8
STOP / SHIFT-STOP	F4 or F10	Shifted function key or double F4
HELP, LAB, DATA, EDIT	F1-F3 and F5, with F6-F9 alternatives	Shifted key or supported double-tap
SUPER / SUB	Up or Page Up / Down or Page Down	Ctrl+P / Ctrl+Y
MICRO / FONT	Ctrl+M / Ctrl+F	Alt variants
SQUARE / ACCESS	Ctrl+Q / Shift+Ctrl+Q	Alt variants
DIVIDE / MULTIPLY / ASSIGN	Ctrl+G / Ctrl+X / Alt+Left	Profile-remappable

The double-tap engine recognizes a second press within roughly 350 ms for selected shifted PLATO actions. Because operating systems reserve different combinations, the active profile and Keyboard Reference should take precedence over memorized platform shortcuts.

4.4 Touch and clipboard

Clicks in the active square are converted to PLATO coordinates and sent as fine-grained touch together with the legacy coarse representation. When curved optical geometry is active, PlatoLives mathematically inverts the visual distortion before reporting coordinates, so appearance never changes the host-visible input contract.

Pasted text is paced to avoid overrunning hosts. Screen copy captures the logical display as an image, while the Live Text Buffer exposes the 64 x 32 text matrix for selection and compact export.

5. Scripting and Session Automation

5.1 Automation that reads the terminal

A useful PLATO script cannot assume that every connection starts on the same page or reaches a prompt after the same delay. PlatoLives scripts inspect the terminal's 64 by 32 character matrix and decide what to do from what is actually displayed. This makes them suitable for login, demonstrations, regression checks and unattended recovery without reducing them to recordings of keystrokes. Execution takes place outside the interface loop, and Cmd+Shift+X or Ctrl+Shift+X stops a running script immediately.

The Script Manager is an editor and library for these procedures. A script has a name, editable source and an optional shortcut or connection-profile association. Attaching a script to a connection profile is the cleanest way to express intent: the profile establishes the host and visual environment, then the script handles the conversation with that host. Scripts remain independent records, so one recovery routine can be reused by several profiles.

5.2 Reading, deciding and acting

The two central instructions are find_text and wait_text. The first examines the current screen immediately; the second waits for literal text or a regular expression to appear, optionally within a bounded rectangle and always with a timeout. Both can return the match coordinates to variables. A robust script combines these tests with labels, conditional branches and counted loops, so every action is justified by a recognized screen state.

Text is sent with send; special terminal actions use key NEXT, key SHIFT-STOP and the other canonical PLATO names. send-env reads a value from the host operating system while redacting it from diagnostic output, which is preferable for passwords. Character, NEXT and command delays can be adjusted when a host requires gentler pacing, but timing supplements state detection rather than replacing it.

5.3 Connection-aware recovery

Version 5.0.0 gives scripts direct control of the network lifecycle. disconnect closes the active session cleanly. reconnect and connect last return to the runtime endpoint; connect default selects the designated profile; connect "Profile Name" chooses a saved profile; and connect host:port addresses a specific endpoint. Recovery should reconnect, wait briefly for protocol synchronization, and resume screen-state detection from a known label. Repeated blind reset keys are less reliable because they assume the very state the script is trying to discover.

5.4 Language essentials

The language is deliberately small: variables contain strings or signed 64-bit integers; labels and goto express recovery paths; if compares values; for ... next handles bounded retries; and wait introduces an asynchronous pause. Comments begin with # or //, and commands and PLATO key names are case-insensitive. Diagnostic instructions can capture screenshots, export all or part of the text matrix, and enable detailed logging. The compact reference belongs in the built-in Scripting Reference Manual; the example below shows how the pieces form a readable operating procedure.

5.6 Example: adaptive login and recovery

```
let username = "demo1"
let group = "demo"
let password = ""

@start:
find_text regex "Press +NEXT +to begin" into x,y
if x >= 0 goto begin
find_text regex "Type +your +CYBIS +name" into x,y
if x >= 0 goto user
find_text regex "Type +the +name +of +your +CYBIS +group" into x,y
if x >= 0 goto group
find_text regex "Enter +your +password" into x,y
if x >= 0 goto password
goto recover

@begin:
key NEXT
goto start
@user:
send username
key NEXT
goto start
@group:
send group
key SHIFT-STOP
goto start
@password:
if password != "" send password
key NEXT
wait 2s
find_text regex "Choose +a +lesson" into x,y
if x >= 0 goto done

@recover:
reconnect
wait 2s
goto start
@done:
exit
```

5.7 Script tools

screenshot, copy-all, copy-area, and diagnostic-log on|off support testing and documentation. Scripts are managed through the Script Manager and may be assigned shortcuts or attached to connection profiles.

6. Rendering Engines and Display Controls

6.1 Choosing a display engine

Mode	Character	Best used for
Crisp Monochrome	Exact, flat monochrome pixels	Protocol and font verification.
Real Plasma	Orange gas-discharge cells with inherent memory and glow	The characteristic PLATO IV presentation.
Split Monochrome	Plasma and crisp views side by side	Immediate optical comparison.
Crisp Color	Exact host color without optical effects	Color-stream verification.
Real Color CRT	Beam, P22 persistence, scanlines, mask, bloom and curvature	CDC 721 / IST-III color presentation.
Real Tektronix 4014	Persistent P31 storage-tube vectors, writing flare and page-erase flash	Tektronix-style vector operation and historical serial pacing.

6.2 Display controls

Real Plasma and Real Color CRT offer appropriate persistence and distortion choices. CRT mode also provides Standard, High and Ultra beam profiles. Tektronix mode provides Instant rendering and serial rates of 110, 150, 300, 600, 1200, 2400, 4800 and 9600 baud. Lower rates make complex pages visibly form as they would across a historical serial link.

The active phosphor theme also coordinates menus, dialogs, telemetry and status indicators. This is a visual integration feature, not a change to terminal data.

6.3 Performance HUD

F12 opens performance and vector telemetry. Depending on the active engine, it can report frame rate, frame time, effective CPU activity, selected display mode and Tektronix queue occupancy. The HUD is diagnostic; it is not required for normal operation.

7. MicroTutor, Audio and Platform Support

7.1 Offline MicroTutor

PlatoLives can run compatible .mte floppy images without a network connection. The local environment combines a Z80/8080 processor core, an emulated CDC IST-II floppy controller and the terminal display. Leaving MicroTutor resets terminal memory, programmable fonts and decoder state before online operation resumes. MicroTutor is unavailable while the Tektronix engine is active because that presentation does not represent the required MicroTutor terminal behavior.

The integrated processor is based on the open-source z80emu engine by Lin Ke-Fong, included with gratitude and kept isolated from unrelated terminal logic.

7.2 Audio

The terminal bell uses non-blocking queued audio at 44.1 kHz, 16-bit mono. Its 880 Hz fundamental, brief harmonic content and shaped envelope provide a compact approximation of the historical terminal sound. Headless environments continue silently when no audio device is available.

7.3 TrueType rendering

Native auxiliary windows on platforms that require software font rasterization use stb_truetype by Sean T. Barrett. This component is used for modern user-interface text and does not replace PLATO's canonical terminal character sets.

7.4 Supported builds

macOS native bundles for Apple Silicon and Intel.
Windows x86-64 and native ARM64 executables using Direct3D 11.
Linux desktop packages for x86-64 and ARM64.
Static Linux console binaries using musl for headless systems, NAS devices and small computers.

8. Menus and Everyday Reference

PlatoLives follows each operating system's menu conventions while preserving the same functional organization. Menu items that depend on an active connection or rendering engine are enabled only when they can perform a meaningful action. Dynamic submenus list the available connection profiles, scripts and keyboard profiles.

C.1 Connection menu

The Connection menu separates two decisions that are often confused: whether to reuse the current terminal window, and which saved profile should define the session. Same window default profile reconnects the current window with the designated default. Same window with profile selects another saved configuration. The corresponding New window commands create an independent terminal, allowing several simultaneous hosts or presentation modes. Disconnect this window closes only the active transport; Close this window removes the terminal itself. Manage Profiles... opens the profile editor.

Task	macOS label or shortcut	Windows and Linux equivalent
Open the default profile in the current window	Same window default profile	Same function in Connection menu
Select a saved profile in the current window	Same window with profile	Same function in Connection menu
Create an independent session	New window default profile, Cmd+N	New window default profile, Ctrl+N
Disconnect the active terminal	Disconnect this window, Cmd+D	Disconnect this window, Ctrl+D
Edit saved profiles	Manage Profiles..., Cmd+,	Manage Profiles...

C.2 Connection Profiles Manager

The Connection Profiles manager stores complete working contexts, not merely server addresses. A profile combines its name, host and TCP port with a rendering engine, optical settings, full-screen preference and an optional startup script. Up to 32 profiles can be stored. The list identifies the available records; selecting one exposes its editable values in the detail area.

Area	Principal controls	Purpose
Identity and network	Profile Name, Host, Port	Names the session and defines the endpoint. The port must be in the range 1-65535.
Display	Display Mode, Phosphor Color, Persistence, Distortion	Saves the visual environment appropriate to that host or use case.
Engine-specific settings	CRT beam and persistence; Tektronix Baud Rate	Appears or becomes relevant only for the selected rendering engine.
Window	Full Screen	Opens the session using the preferred presentation state.
Automation	Enable Startup Script, Startup Script	Runs an embedded script after connection; script bodies are limited to 16 KB.

Create a profile with Add Profile or the plus button, then edit the selected record. Save Profiles commits the list to profiles.ini in the platform's PlatoLives configuration directory. Connecting with a profile closes the current socket cleanly, applies the saved optical state and starts the new transport. Changing between color and monochrome capabilities may require terminal capability renegotiation.

C.3 Keyboard Profiles Manager

Keyboard profiles translate physical keys into canonical PLATO actions without changing the protocol seen by the host. The manager shows built-in and custom profiles in one list; the active profile is marked. Built-in definitions are protected, but any built-in profile can be duplicated and adapted. Custom profiles are saved as individual JSON files and become active immediately, without reconnecting.

Control	Function
New Profile / +	Creates a new custom profile.

Control	Function
Duplicate	Makes an editable copy of the selected profile, including a built-in one.
Delete	Removes a custom profile; unavailable for built-in profiles.
Binding table	Shows the physical key, modifiers and resulting PLATO action.
Press Key to Bind...	Captures the next physical key-down event and modifier combination.
PLATO Action	Selects the canonical action or raw character produced by the binding.
Set as Active Profile	Applies the selected mapping to the live terminal.

Built-in profile	Intended control style
Standard PLATO	Default terminal and lesson operation.
WASD Gaming	W, A, S and D directional movement.
WASX / Avatar	Classic dungeon control with W forward, X back, A left and D right.
Numpad Arrows	Eight-direction movement on the numeric keypad.
WADX	Traditional TUTOR directional diamond.

Use Cmd+Shift+G on macOS or Ctrl+Shift+G on Windows and Linux to cycle to the next preset.

C.4 Keyboard Reference

Show Keyboard Reference opens the static PLATO Keyboard Reference card. It documents the application's default canonical shortcuts, including NEXT, BACK, HELP, LAB, DATA and STOP, together with application shortcuts such as F11 for full screen and F12 for the Performance HUD. It does not display or follow the active custom keyboard profile. Use the Keyboard Profiles Manager when the current profile's bindings must be inspected or changed.

C.5 Script Manager and scripting reference

The Script Manager maintains named automation scripts in a compact editor. A script can be created, cloned, deleted, assigned a shortcut and run against the active session. The editor accepts bodies up to 16 KB and the script engine supports up to 128 variables. Saved scripts appear dynamically in the Scripts menu. Execution is asynchronous; Cancel Script Execution or Cmd+Shift+X on macOS and Ctrl+Shift+X elsewhere stops a running script.

Control	Function
New	Creates an empty script.
Clone	Duplicates the selected script as a starting point.
Delete	Removes the selected script after confirmation.
Script Name	Sets the menu and library name, up to 64 characters.
Shortcut / Hotkey	Captures an optional key combination for direct execution.
Script editor	Holds the script body in a monospaced editing area.
Run Script	Starts the selected script in the active terminal session.
Cancel Execution	Requests immediate termination of the running script.

Scripting Reference... opens the built-in command reference. Chapter 5 explains the state-aware workflow; the reference should be used for exact syntax and parameters.

C.6 Live Text Buffer and copy tools

Show Live Text Buffer exposes the terminal's 64-column by 32-row logical character grid in real time. Each cell retains both its character and PLATO character-set identity. The tool is useful when text must be inspected independently from plasma, CRT or Tektronix presentation. PLATO's bottom-left display coordinates are converted so that text row zero appears at the top of the buffer.

Copy command	Result
Copy All Text Verbatim	Preserves horizontal placement while removing trailing spaces at the end of each line.
Copy All Text Compact	Removes leading and trailing margins and compresses repeated spaces or tabs.
Copy Selected Text...	Copies a user-selected rectangular character area.
Copy Screen Image	Copies the rendered screen as an image rather than extracting characters.
Paste Text / Cancel Paste	Sends paced clipboard text or stops a paste already in progress.

C.7 Diagnostic tools

Diagnostic facilities are intentionally separate because they answer different questions. Enable Diagnostic Log records network and protocol activity. Enable Renderer Performance Log records optical-renderer telemetry. Enable Z80 Diagnostic Log, under Micro Tutor, traces the local processor environment. Values sent with the scripting command send-env are replaced by [REDACTED] in diagnostic output.

C.8 Performance and Vector HUD

F12 toggles the telemetry display. On macOS it is integrated as an overlay; Windows and Linux use a dedicated auxiliary presentation. The label changes between Show and Hide according to the current state.

Field	Meaning	Availability
USER	Connected session identity or STANDALONE while offline.	All modes
FPS TOTAL	Composite presentation rate.	All modes
DISPLAY MODE	Active rendering engine.	All modes
CPU	Optical-engine processing load.	All modes
FPS PARTIAL / FPS FULL	Dirty-region and full-frame activity.	Plasma and CRT
BAUD	Selected emulated serial speed.	Tektronix only
TEK QUEUE	Ring-buffer occupancy as count and percentage of 8192 events.	Tektronix only

C.9 View, Micro Tutor and platform-specific tools

The View menu selects Crisp Monochrome, Split Monochrome, Real Plasma, Crisp Color, Real Color CRT or Real Tektronix 4014. Engine-specific submenus expose only meaningful persistence, beam, distortion and serial-speed controls. Override Color offers None, Amber, Green and White outside Tektronix mode. F11 toggles full screen. Linux also provides Hide Menu Bar; macOS uses the native system menu bar and offers native window tabs.

The Micro Tutor menu loads and boots .mte images in the integrated Z80 environment. MicroTutor controls are disabled while Real Tektronix 4014 is active because that engine does not represent the terminal behavior required by the local environment.

9. Diagnostics, Performance and Troubleshooting

9.1 Diagnostic logging

The diagnostic log records session headers, stream direction and offsets, hexadecimal and printable data, protocol events, outgoing keys, renderer changes, profile selections, and relevant optical state. Sensitive data sent through send-env is redacted.

9.2 Performance strategy

The application renders actively when data, animation or phosphor decay requires work. Once a static optical image has settled and no region is dirty, periodic presentation stops. Rendering restarts immediately on network data, input, exposure, mode change or profile change.

The Tektronix engine decouples network arrival from progressive drawing through an 8192-event queue and a baud-credit pacer. This prevents a slow visual rate from blocking transport handling.

9.3 Troubleshooting

Symptom	Likely cause	Recommended action
Connection fails or times out	Host, port or network unavailable	Review the selected profile; Retry or select another profile.
Connection attempt appears unwanted	Wrong profile or offline host	Press Escape to cancel immediately.
Color content is monochrome	Monochrome engine or monochrome profile selected	Choose Crisp Color or Real Color CRT and reconnect if negotiation must restart.
Keys do not match the lesson or game	Inappropriate keyboard profile	Select Standard PLATO or the intended gaming profile; review bindings.
A script acts on the wrong screen	Blind timing or overly broad match	Use wait_text with a bounded area and timeout; add a recovery label.
A script cannot recover	Connection state is stale	Use reconnect or connect last, wait for synchronization, then restart state detection.
Tektronix page appears too quickly	Instant speed selected	Choose a historical baud rate.
Tektronix drawing falls behind	Very low baud rate or exceptionally large page	Raise the rate or use Instant; inspect queue occupancy with F12.
Curved image but inaccurate touch	Stale profile or geometry state	Reapply the distortion mode or reconnect.
Long luminous trail	Persistence too high	Select a shorter plasma or CRT persistence setting.
MicroTutor image does not start	Unsupported image or Tektronix mode active	Select a PLATO display engine and reopen the .mte image.
No bell is heard	No available audio output	Check the OS/SDL audio device; headless use is intentionally silent.

10. Technical Architecture and Build Notes

10.1 Shared core

The shared core is written in C11 and contains transport-independent protocol decoding, terminal state, framebuffers, graphics primitives, canonical fonts, text extraction, touch conversion, keyboard actions, script execution and optical state. Platform front ends provide windows, dialogs, file selection, graphics presentation, menus and native event handling.

10.2 Transport

Sockets are non-blocking. Connection attempts use a bounded timeout and an explicit cancellation flag. Runtime endpoint state supports reconnect and connect last without temporary storage. Network input is buffered separately from display consumption.

10.3 Versioning and builds

The root VERSION file is the single source for the public version. Build steps generate plato_version.h and synchronize application resources and About dialogs. The universal build pipeline coordinates Clang and Zig targets for macOS, Windows and Linux, including musl-linked console binaries.

10.4 Configuration storage

Connection profiles and scripts use INI-based configuration. Keyboard profiles use JSON. User-interface code treats these records as versioned application data and preserves unknown or inapplicable settings where possible so that switching display engines does not destroy another engine's calibration.

11. License, Credits and References

PlatoLives is released under the Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International license (CC BY-NC-SA 4.0).

Project author: Fabio Montarsolo

Acknowledged projects and upstream components

Pterm and its contributors, for preserved knowledge of PLATO protocols, PTAT keyboard translation, typography and live-host behavior.
z80emu by Lin Ke-Fong, used by the local MicroTutor environment.
stb truetype by Sean T. Barrett, used for TrueType rasterization in _ supporting interfaces.
Cyber1 operators, contributors, lesson authors, archivists and testers.

Version 5.0.0 release summary

Version 5.0.0 consolidates PlatoLives as a multi-platform PLATO environment centered on reusable profiles and observable terminal behavior. Its most important advances are universal keyboard profiles, resilient asynchronous connections, network-aware stateful scripting, the Tektronix 4014 DVBST engine with historical serial pacing, offline MicroTutor operation, and native support across modern macOS, Windows and Linux systems. These additions extend the application without changing the authoritative 512 x 512 PLATO logical model.

Appendix A. Real Plasma Rendering: Technical Design

A.1 Historical and logical model

The original PLATO IV display was an AC gas-discharge neon plasma panel with inherent bistable memory. PlatoLives separates logical state from optical presentation. Protocol decoding, vectors, blocks and font blits operate on the authoritative 512 x 512 bitboard; the optical stage expands that state to a continuous 2048 x 2048 surface.

A.2 Four-by-four cell footprint

```
[ 0.65  0.82  0.82  0.65 ]
[ 0.82  1.00  1.00  0.82 ]
[ 0.82  1.00  1.00  0.82 ]
[ 0.65  0.82  0.82  0.65 ]
```

A stable per-cell gain in the interval 0.98 to 1.02 reproduces fixed panel variation without temporal noise.

A.3 Neon persistence

$E(t + dt) = E(t) \exp(-dt / tau)$, $tau = T_{decay} / 10$

When energy falls below 0.0001 it is clamped to zero. The tile then leaves the active set unless another logical or optical dependency remains. This strict terminal condition is what allows a settled screen to become computationally idle.

A.4 Dual separable blur

A local radius of four supersampled pixels models inter-cell diffusion. A wider radius of twelve models scattering through the front glass. Both two-dimensional box blurs are separated into horizontal and vertical running-sum passes:

```
S(x) = S(x-1) + source[x+R] - source[x-1-R]
dest[x] = S(x) / (2R+1)
```

The cost per destination sample is independent of radius. Dirty work is bounded by 16 x 16 logical tiles and expands to neighboring tiles only when the optical kernel can cross a tile boundary.

A.5 Density fusion and orange response

```
n = clamp((local_glow - 0.25) / 0.40, 0, 1)
fused = core + 0.82 n (1 - core), for core > 0.05
```

The fused core, local glow and wide glow are combined through a restrained orange-biased response. The goal is the dense, warm emission of a gas-discharge panel, not a generic soft-focus effect.

A.6 Distortion and touch

Real Plasma may be flat, cylindrical or barrel-distorted. Cylindrical mode bends only the horizontal axis; barrel mode bends both axes with restrained coefficients. Pointer input passes through the inverse transform before conversion to PLATO coordinates. Split Monochrome remains flat to provide a valid side-by-side comparison.

Appendix B. Real Color CRT and Optical Geometry: Mathematical Design

B.1 Three-tap Gaussian electron beam

For a logical sample centered at x0, the beam contributes to three adjacent supersample positions i in {-1,0,+1}. With a selectable sigma of 1.8, 2.4 or 2.8:

```
g_i = exp(-(i^2) / (2 sigma^2))
w_i = g_i / (g_-1 + g_0 + g_+1)
C_beam(x0+i,y) += gain * w_i * C_logical(x0,y)
```

The normalized kernel conserves line energy before profile gain, scanline modulation, mask modulation and additive bloom.

B.2 Pure chromatic shadow mask

The repeating four-position mask is indexed by supersample x modulo four. Channel-selective boosts represent red, green and blue sites, followed by a neutral matrix-wire position. Profile calibration normalizes aggregate luminous energy, so the mask does not impose artificial light loss; non-blue sites have no constructed blue crosstalk.

B.3 Soft analog scanlines

For supersample row y_s, the logical cathode phase is:

```
phi = 2 pi (y_s mod 4) / 4
m_scan(y_s) = 1 - d * (1 - cos(phi)) / 2
C_scan = m_scan * C_mask
```

The groove depth d is 0.22, 0.20 or 0.18 for the Standard, High and Ultra beam profiles. Cylindrical distortion leaves y unchanged, so scanlines remain straight.

B.4 Calibrated forward distortion

For normalized undistorted coordinates x,y in [-1,1], cylindrical projection uses k = 0.018 only on x:

```
x_d = x (1 + 0.018 x^2)
y_d = y
```

Barrel projection uses separate coefficients that reflect the 4:3 tube geometry:

```
r2 = x^2 + y^2
x_d = x (1 + 0.018 r2)
y_d = y (1 + 0.012 r2)
```

Samples outside the curved aperture are black.

B.5 Inverse touch mapping

Cylindrical inversion solves x_d = x(1+kx^2) by Newton iteration while leaving y unchanged. Barrel inversion repeatedly removes the radial scale from both coordinates. The coefficients are deliberately small, so a few bounded iterations converge across the usable aperture. Visual geometry therefore never changes the host-visible touch contract.

B.6 Continuous P22 color-phosphor decay

Host-selected logical color and luminous energy are stored separately:

```
E(t + dt) = E(t) exp(-dt / tau),  tau = T_CRT / 10
C_emit(t) = L * E(t)
```

Below 0.0001, energy and emitted color are set to zero. The threshold is visually negligible and supplies the exact end state required for idle quiescence.

B.7 Additive halation and profile composition

```
C_out = clamp(gain * C_scan + bloom * (B_4(C_scan) + B_12(C_scan)), 0, 1)
```

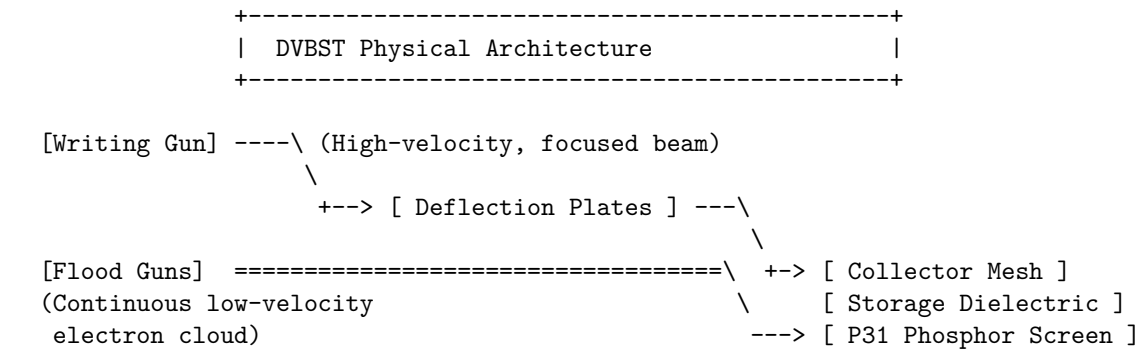
B_4 and B_12 are separable box blurs. Bloom remains chromatic and additive, representing light scattered in smoked front glass rather than desaturation toward white.

Appendix C. The Tektronix 4014 Optical Engine & Vector Architecture

The following technical chapter is reproduced verbatim from the supplied source.

C.1 Operating Physics of the Direct-View Bistable Storage Tube (DVBST)

To emulate the Tektronix 4014 terminal with historical and optical accuracy, one must first discard conventional raster-scan CRT paradigms (such as scanlines, refresh cycles, and shadow masks). The Tektronix 4014 does not rely on a dynamic refresh buffer; its display architecture is governed entirely by the physical properties of the **Direct-View Bistable Storage Tube (DVBST)**.



C.1.1 The Dual-Gun Electron Optics

A DVBST contains two distinct electron beam systems operating in tandem within an evacuated glass envelope:

1. **The Focused Writing Gun:**
   An electrostatically deflected, high-energy electron gun producing a narrow, high-velocity electron beam. When unblanked, this beam is directed across the screen surface by high-voltage deflection plates driven by 10-bit or 12-bit Digital-to-Analog Converters (DACs). Its purpose is not to excite visible light directly for human observation, but to deposit localized electrostatic charge patterns onto a target storage mesh.
2. **The Flood Guns:**
   A pair of low-velocity electron emitters positioned symmetrically at the perimeter of the funnel section of the CRT. Unlike the writing gun, the flood guns emit an unfocused, continuous cloud of low-energy electrons covering the entire surface of the display. Collimating electrodes inside the tube shape this electron cloud so that it approaches the display faceplate in parallel, uniform trajectories.

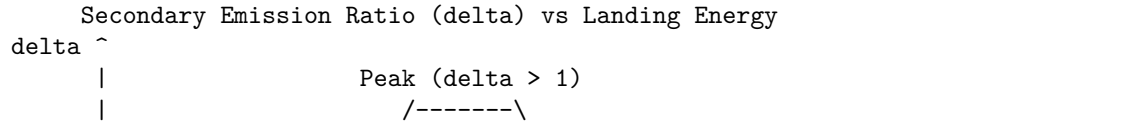
C.1.2 Secondary Emission and Dielectric Bistability

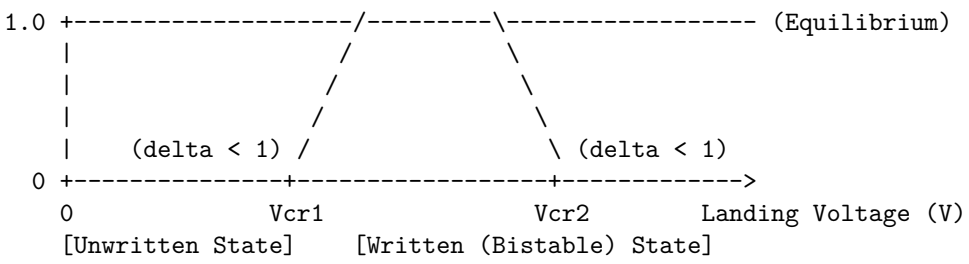
The heart of the DVBST is the target assembly, positioned directly behind the faceplate. It consists of a conductive collector mesh and a thin, discontinuous dielectric layer (often granular zinc orthosilicate or silicon oxide) deposited directly over the phosphor layer.

The physical behavior of this dielectric target depends entirely on the **Secondary Electron Emission Ratio (δ)**, defined as the average number of secondary electrons knocked out of the material per primary incident electron:

$$\delta = \frac{I_{\text{secondary}}}{I_{\text{primary}}}$$

The ratio δ is a non-linear function of the incident electron velocity (landing energy): * At very low landing voltages (below the first crossover potential $V_{\text{cr1}} \approx 50$ V), $\delta < 1$. Electrons accumulate on the dielectric, charging it negatively toward the cathode potential of the flood guns. Once negatively charged, the surface repels further flood-gun electrons. This corresponds to the **Unwritten (Extinguished) State**. * When the high-velocity **Writing Gun** strikes the dielectric, the landing energy exceeds V_{cr1} into the region where $\delta > 1$. The incident beam knocks out more secondary electrons than it injects. These secondary electrons are collected by the positively biased collector mesh. Consequently, the written area loses net negative charge and becomes **electrostatically positive**. * When the writing beam departs, the **Flood Gun electrons** reach this positively charged region. Because the surface is now positive, the flood electrons strike it with sufficient velocity to keep $\delta \geq 1$, knocking out secondary electrons and permanently clamping the region in its positive equilibrium state (the **Written State**). As these flood electrons pass through the dielectric target, they continuously excite the underlying **P31 phosphor layer**, producing continuous luminescence without requiring memory refresh cycles.





C.1.3 Deflection Settling Times and Vertex Dwell

The deflection plates of a 19-inch storage tube operate at high electrostatic voltages. When a vector is drawn, the deflection amplifiers require finite settling times ($\tau_{\text{settle}} \approx 10$–$20\,\mu$s) to stabilize at the new coordinate before the beam is blanked or redirected.

During these transient pauses at line vertices, the electron beam dwells on a single point while the deflector voltages settle. This elevated **dwell time** delivers a significantly higher integrated electron dose to the dielectric target, creating visible, intense, slightly blooming **dwell spots** at the endpoints of every drawn line.

C.1.4 The Page Erase Flash (Bulk Dielectric Discharge)

Because the dielectric target is macroscopic and contiguous, individual stored charge packets cannot be electrostatically repelled or selectively erased without disturbing adjacent regions.

To clear the screen, the terminal must perform a **Page Erase (Bulk Erase)**: 1. A positive voltage pulse is applied to the collector mesh, driving the entire dielectric surface above V_{cr1}. Flood-gun electrons are pulled in en masse across the entire screen, triggering universal secondary emission: **the entire phosphor screen detonates into a brilliant, blinding green flash**. 2. Immediately following the flash, the collector mesh voltage is driven negative below the flood-gun cathode potential. The flood electrons flood the dielectric, driving the entire target below V_{cr1} back into the negative cutoff state. 3. The stored charge is completely dissipated, returning the screen to black in approximately 400 ms.

C.2 Technical Challenges in Achieving Believable Simulation

Simulating a DVBST terminal inside a modern graphics engine presents distinct engineering challenges:

C.2.1 Vector Continuity vs. Discrete Pixel Matrices

Modern rendering pipelines (DirectX, Metal, Vulkan) are optimized for textured polygon rasterization across discrete 2D pixel grids. The Tektronix beam, conversely, is a continuous analog physical phenomenon. Simulating smooth, sub-pixel vector trajectories without digital stair-stepping (aliasing) while maintaining the characteristic beam thickness (~0.4 mm spot size on physical hardware) requires high-resolution sub-pixel spatial integration.

C.2.2 Transient Optical Dynamics (The Phosphor Bloom Paradox)

The visual experience of a Tektronix 4014 consists of two superimposed temporal domains: * **The Transient Writing Beam Flare:** A piercing, high-intensity fluorescent flash occurring directly under the focused writing beam. Due to local thermoluminescence and phosphor overload, this core saturates beyond the linear response of the human eye, appearing intensely white-hot with an emerald corona. * **The Stored State:** A steady, dimmer, persistent green luminescence maintained indefinitely by the diffuse flood guns. * **Flood Gun Dark Luminescence:** The dark areas of the tube are never pure digital black; continuous bombardment by flood electrons induces a low-intensity, diffuse green-gray phosphorescent background.

Accurately capturing this behavior requires an optical engine capable of combining transient exponential decays with non-linear colorimetric response curves.

C.2.3 The Baud-Rate Serial Bottleneck

On historical systems, vector coordinates were not loaded instantaneously into GPU memory; they arrived across an RS-232 serial link operating at rates between 110 and 9600 baud. The gradual, mechanical unfolding of diagrams—where complex curves visibly trace across the screen and characters pop into existence byte-by-byte—is integral to the physical

authenticity of the terminal. Emulation must throttle rendering based on physical bit budgets without introducing timing jitter or CPU stalls.

C.3 PLATO vs. Tektronix: Architectural and Semantic Divergence

A central architectural dilemma in PlatoLives is the radical divergence between **PLATO terminal semantics** and **Tektronix hardware mechanics**.

PLATO Plasma Panel (Digivue)	Tektronix 4014 (DVBST)
512 x 512 Discrete Neon Gas Discharge Cells	4096 x 3120 Continuous Analog Mesh
Inherent Per-Cell Memory	Macroscopic Target Secondary Emission
Selective Cell Ignition & Erasure	ZERO Selective Erase Capability (Page Erase Flash Only)
Destruction leaves clean, severed coordinate gaps	Overwriting causes charge accumulation or blooming smudges

C.3.1 The PLATO Plasma Cell Mechanics

The native PLATO terminal (the Owens-Illinois Digivue 512-60 panel) consists of an array of 512×512 discrete neon-argon gas discharge cells insulated between glass dielectric sheets. * Each cell has its own sustained bistable discharge driven by alternating high-frequency sustain pulses. * An individual pixel can be ignited (`WRITE`) or extinguished (`ERASE`) by addressing its specific horizontal and vertical electrode wires. * **Destructive Overwriting:** When a moving object (such as a sprite in a TUTOR flight simulator or an interactive diagram) passes across static graphics and is erased using PLATO's `ERASE` or `INVERSE` mode, **the underlying pixels are physically extinguished**. If a vector crosses an axis and is erased, the axis is severed, leaving a physical void.

C.3.2 The DVBST Erase Impossibility

As established in Section C.1.4, a physical Tektronix 4014 cannot selectively turn off stored charge. When a PLATO system transmitted a vector in `ERASE` mode to a real Tektronix terminal: * The Tektronix hardware simply ignored the erase attribute or traced over the line with an unblanked beam, adding *more* electrostatic charge to the mesh and exacerbating phosphor bloom. * The only method to update an interactive scene on a physical 4014 was for the mainframe to issue a full `Page Erase` (400 ms flash) and re-transmit every surviving vector across the serial line at 1200 or 9600 baud.

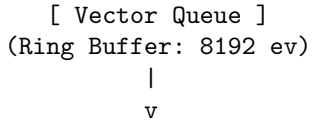
C.3.3 The "Display List" Fallacy

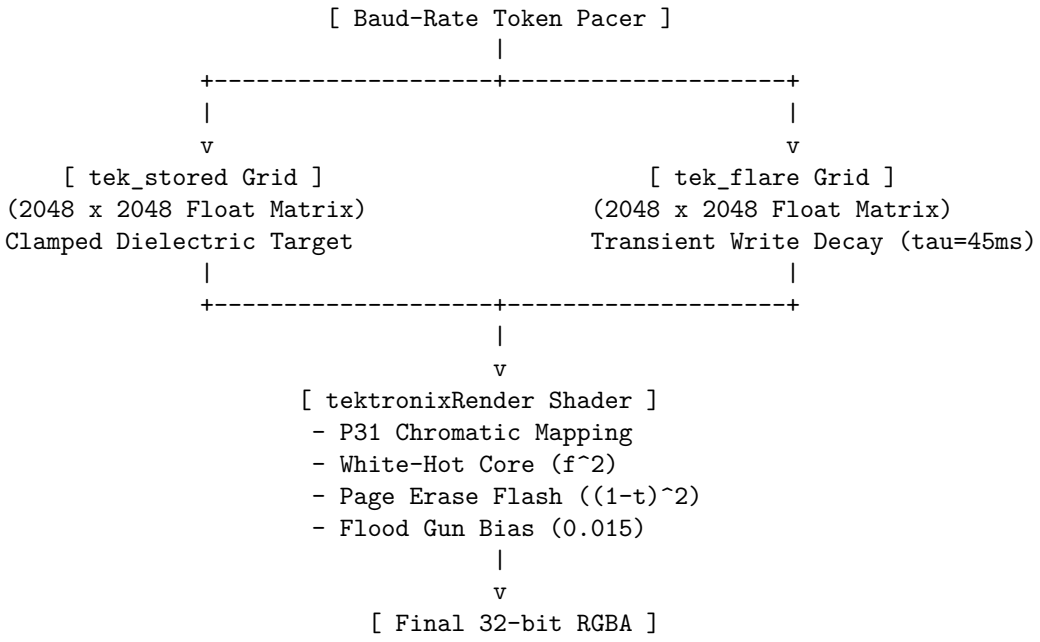
During the architectural design of PlatoLives, an alternative proposal was evaluated: *maintain an in-memory vector Display List (a dynamic list of drawn line primitives), remove lines upon receiving an erase command, and redraw the screen cleanly every frame.*

This proposal was rejected as an **anachronistic anti-pattern**: 1. **Falsification of PLATO Physics:** Reconstructing intersected lines severed by PLATO `ERASE` commands creates a "too-clean" vector presentation that never existed on the original hardware. TUTOR programmers engineered algorithms around the destructive behavior of the plasma panel; repairing these gaps via display lists breaks historical fidelity. 2. **Falsification of Tektronix Dynamics:** Display lists enable instantaneous screen clearing without the characteristic secondary emission flash and eliminate the physical memory effects of the storage mesh.

C.4 The PlatoLives Optical Engine Architecture

To reconcile physical realism, destructive semantics, and real-time 60 FPS performance, PlatoLives implements a multi-tier **Continuous Decoupled Optical Architecture**.





C.4.1 Dual-Grid Spatial Decoupling (`tek_stored` & `tek_flare`)

Rather than relying on post-process spatial convolution passes (e.g., iterative box or Gaussian blurs over the entire screen), the engine decouples the physical state into two high-density floating-point matrices allocated at 2048×2048 **resolution** ($4\times$ oversampling relative to PLATO's 512×512 coordinate space):

```
/* In src/plato_optical.c */
float *tek_stored;  /* 2048 x 2048 x sizeof(float) = 16 MB: Dielectric Target */
float *tek_flare;   /* 2048 x 2048 x sizeof(float) = 16 MB: Transient Photon Flux */
```

1. **`tek_stored` (Dielectric Charge):**
   Simulates the bistable target mesh. Values represent normalized electrostatic potential clamped strictly to $[0.0, 1.0]$. Charge stored here does not decay; it represents the sustained illumination driven by the flood guns.
2. **`tek_flare` (Transient Kinetic Energy):**
   Simulates the instantaneous photon release from high-energy electron impact. It accumulates energy linearly without clamping and undergoes aggressive temporal exponential decay in the compositing loop:

$$\text{flare}(t + \Delta t) = \text{flare}(t) \cdot \exp\left(-\frac{\Delta t}{\tau_{\text{flare}}}\right), \quad \tau_{\text{flare}} = 0.045 \text{ s}$$

C.4.2 Continuous Sub-Pixel Splatting & Settling Emulation

Vectors are rasterized into both matrices via `plato_tek_splat_line()` using fractional sub-pixel advancement:

```
/* Sub-pixel step calculation */
float step = 0.8f;
int steps = (int)ceilf(dist / step);
float x_step = dx * (1.0f / (float)steps);
float y_step = dy * (1.0f / (float)steps);
```

At each fractional step, a compact 2D Gaussian kernel ($\sigma = 1.0$, $r = 2$ sub-pixels) distributes energy into the matrices:

$$w = \exp\left(-\frac{\Delta x^2 + \Delta y^2}{2\sigma^2}\right)$$

```
row_f[px] += intensity * w;                    /* Uncapped flare */
float s_val = row_s[px] + (intensity * w) * 0.7f;
row_s[px] = (s_val > 1.0f) ? 1.0f : s_val;     /* Bistable clamping */
```

Dwell Spot Injection To emulate deflector amplifier settling times (Section C.1.3), the line rasterizer explicitly injects high-gain dwell spots at the vector endpoints prior to step interpolation:

```
/* Vertex Dwell Time: High-gain splat at line origins and terminations */
plato_tek_splat_point(flare, stored, width, height, x0, y0, 0.9f, 1.6f, 0.7f);
plato_tek_splat_point(flare, stored, width, height, x1, y1, 0.9f, 1.6f, 0.7f);
```

C.4.3 The Erase Resolution: Trench Carving vs. Canonical Resync

PlatoLives solves the historical erase conflict through an explicit dual-mode mechanism:

1. Geometric Vector Erase (Destructive Trench Carving) When an individual line vector is processed with `PLATO_SCREEN_ERASE` or `PLATO_SCREEN_INVERSE`, the engine traverses the vector path and clears a discrete bounding radius ($r = 3$ sub-pixels) in `tek_stored`:

```
for (int py = by0; py <= by1; py++) {
    float *row_s = p->tek_stored + py * PLASMA_WIDTH;
    for (int px = bx0; px <= bx1; px++) row_s[px] = 0.0f;
}
```

This physically carves a void through the stored dielectric charge. If this path crosses an existing vector, **the intersection is severed permanently**. No display list reconstructs it, preserving the historical behavior of destructive storage media.

2. Text and Block Resync (`plato_tek_resync_box_from_fb`) For character glyphs and block erasures, carving arbitrary circular sweeps introduces severe visual distortion when fonts are updated in place. Here, the engine leverages the canonical 512×512 1-bit terminal framebuffer (`term->fb`):

```
/* Estendi di 1 pixel PLATO (4 subpixel) per assorbire l'alone gaussiano */
int px_min = (x0 > 0 ? x0 - 1 : 0) * PLASMA_SCALE;
int px_max = (x1 < PLATO_WIDTH - 1 ? x1 + 2 : PLATO_WIDTH) * PLASMA_SCALE - 1;
...
/* 1. Azzeramento locale chirurgico del dielettrico */
for (int py = py_min; py <= py_max; py++) {
    for (int px = px_min; px <= px_max; px++) {
        row_s[px] = 0.0f; row_f[px] = 0.0f;
    }
}
/* 2. Ricostruzione perfetta dei soli punti realmente accesi in fb */
for (int py = scan_y0; py <= scan_y1; py++) {
    for (int px = scan_x0; px <= scan_x1; px++) {
        if (plato_fb_get_pixel(fb, px, py)) {
            plato_tek_splat_point(...);
        }
    }
}
```

The engine clears the character bounding box plus an expanded 1-PLATO-pixel (4 sub-pixel) halo margin to extinguish residual optical bloom, then re-splats only the active bits recorded in the canonical framebuffer.

C.4.4 Hardware Serial Pacer (Token-Bucket Flow Control)

Vector events are queued into an 8192-slot ring buffer (`term->vector_queue`). Consumption is metered by a bit-rate accumulator driven by wall-clock time Δt:

$$\text{credit_bits} \mathrel{+}= \Delta t \times \text{baud}$$

$$\text{credit_bits} \leq \text{baud} \times 0.250\ \text{s} \quad \text{(Anti-Burst Clamp)}$$

Each queued event carries an architectural bit cost corresponding to 10-bit asynchronous framing (1 start bit, 8 data bits, 1 stop bit):

$$\text{Cost(Vector Line)} = 4\ \text{bytes} \times 10\ \text{bits} = 40\ \text{bits}$$

$$\text{Cost(Character)} = 1 \text{ byte} \times 10 \text{ bits} = 10 \text{ bits}$$

$$\text{Cost(Page Erase)} = 2 \text{ bytes (ESC FF)} \times 10 \text{ bits} = 20 \text{ bits}$$

If the accumulated credit is less than the event cost, the consumption loop terminates for the current frame. Vector processing pauses cleanly mid-path, allowing lines to visibly trace across the screen at rates authentic to 300, 1200, or 9600 baud.

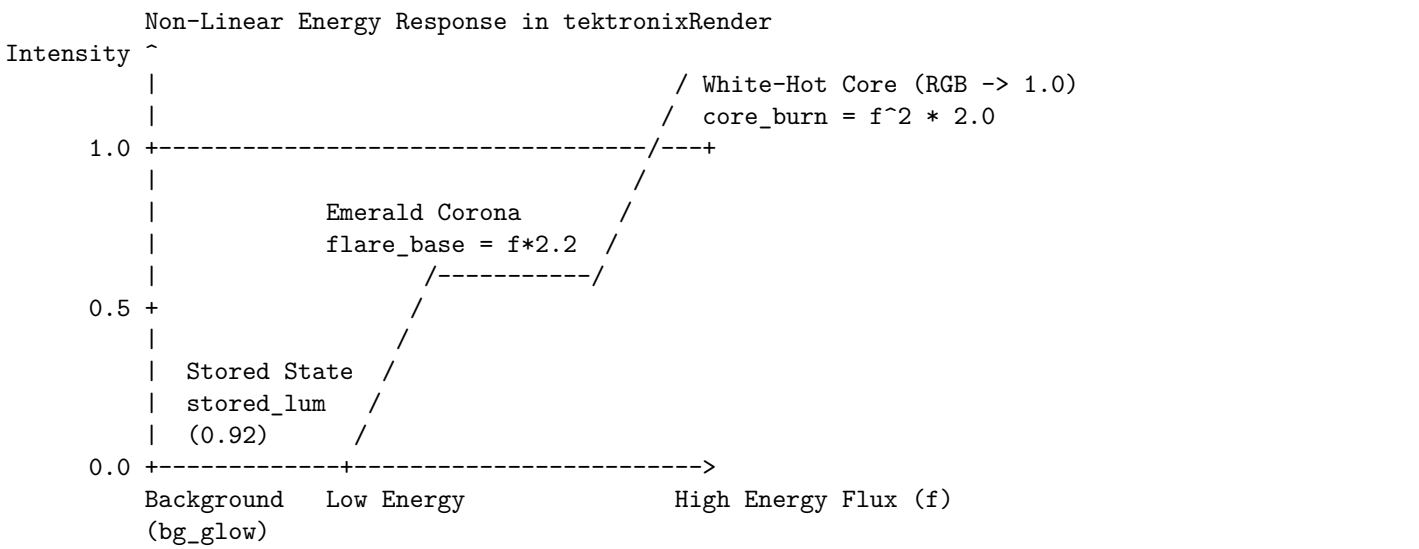
C.4.5 On-the-Fly Non-Linear Chromatic Synthesis

The compositing stage (`tektronixRender`) executes across the 2048×2048 output buffer in a single pass without secondary convolution buffers:

```
/* Stored luminescence */
float stored_lum = (s > 1.0f ? 1.0f : s) * 0.92f;

/* Thermoluminescent over-saturation model */
float flare_base = f * 2.20f;         /* Linear green P31 halo */
float core_burn  = f * f * 2.00f;     /* Quadratic white-hot core */

/* P31 Spectral Synthesis */
float r = bg_glow * 0.3f + stored_lum * 0.16f + flare_base * 0.30f + core_burn + erase_flash * 0.35f;
float g = bg_glow * 1.0f + stored_lum * 0.96f + flare_base * 1.00f + core_burn + erase_flash * 0.98f;
float b = bg_glow * 0.4f + stored_lum * 0.30f + flare_base * 0.10f + core_burn + erase_flash * 0.40f;
```



1. **Flood Gun Bias (`bg_glow`):**
   A base optical offset of 0.015 with green spectral weighting $(0.3, 1.0, 0.4)$ ensures unwritten areas exhibit the authentic dark phosphorescent floor of a working tube.
2. **Non-Linear Core Burn (`core_burn`):**
   Because intense electron flux saturates the chemical excitation states of the P31 phosphor dopants, the color output shifts non-linearly. The quadratic term $f^2 \times 2.0$ drives all three RGB channels toward 1.0, producing an incandescence where the center of the vector burns **white-hot**, bordered by an emerald green fringe.
3. **Erase Flash Synthesis (`erase_flash`):**
   Triggered by `VEC_EVENT_ERASE`, a quadratic temporal curve clears stored charge while washing the screen in high-luminance P31 green:

$$\text{flash}(t) = \left(1.0 - \frac{t_{\text{elapsed}}}{0.400 \text{ s}}\right)^2, \quad t_{\text{elapsed}} < 0.400 \text{ s}$$

C.5 Architectural Conclusions & Verification

The PlatoLives Tektronix optical pipeline resolves the conflict between legacy hardware constraints and modern rendering systems:

- **Pure State-Based Simulation:** Zero reliance on anachronistic display lists preserves the physical, destructive reality of historical displays.
- **Bounded Working Footprint:** By replacing multi-pass spatial box blurs with coupled sub-pixel grids (`tek_stored` + `tek_flare`), the entire simulation runs in a compact 48 MB footprint, maintaining a steady 60 FPS on CPU without requiring dedicated compute-shader hardware.
- **Unified Optical Accuracy:** Sub-pixel Gaussian splatting, vertex dwell timing, token-bucket serial throttling, and quadratic phosphor saturation combine to deliver an authentic recreation of the Tektronix 4014 storage tube within the PLATO software ecosystem.

Appendix D. Historical and Technical References

Tom Hunter, Paul Koning, Joe Stanton, Bill Galcher, Steve Zoppi and Dale Sinder, *Pterm: A PLATO Terminal Emulator*; FrameCanvas.cpp and ptermkeytabs.h.
Jack Stifle, *The PLATO IV Architecture*, CERL Report X-20, University of Illinois, May 1971, revised August 1972.
Jack Stifle, *The PLATO IV Terminal: Description of Operation*, University of Illinois, revised August 1974.
Control Data Corporation, *CDC IST-III Terminal Reference Manual*, publication 84002390.
Brian Dear, *The Friendly Orange Glow: The Untold Story of the PLATO System and the Dawn of Cyberculture*, Pantheon Books, 2017.
Donald L. Bitzer and H. Gene Slottow, “The Plasma Display Panel - A Digitally Addressable Display with Inherent Memory,” AFIPS, 1966.
Michael L. McMahan, *The Application of Plasma Panel Technology to Graphic Display Terminals*, University of Illinois, January 1972.
Tektronix 4014 storage-display technical literature and operating documentation.
PlatoLives 5.0.0 source repository and technical architecture snapshot, October 2026.
Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International Public License, SPDX CC-BY-NC-SA-4.0.